Rapport Technique

TP Examen 1

Gestion d'une ligne de Commande

Groupe :	25
Membres :	Ngomde Cadet Kamdaou
	Bediang Leandrine Orline
Formation :	Master DI
Institution :	Université des Montagnes
Date :	Octobre 2026

Table des matières

1 Présentation du projet **2**

2 Architecture de la base de données **2**
2.1 Choix techniques 2

3 Architecture applicative **2**
3.1 Organisation des fichiers 2
3.2 Pattern architectural 2

4 Sécurité **3**

5 Interface utilisateur **3**
5.1 Technologies front-end 3
5.2 Choix d'ergonomie 3

6 Tableau de bord – Analyse de performance **4**

7 Gestion des transactions **4**

8 Instructions de déploiement **4**

Présentation du projet

Ce projet implémente une application web de gestion de commandes en PHP/MySQL. Elle permet la création, consultation, modification et suppression de produits, clients et commandes, avec un tableau de bord statistique pour l'analyse de performance des lignes de commande.

L'application répond aux cinq tâches du sujet d'examen :

1. Formulaire d'enregistrement des produits d'une commande
2. Affichage des listes (produits, commandes, clients)
3. Modification d'un produit
4. Suppression d'un produit dans une commande
5. Tableau de bord avec statistiques sur les lignes de commande

Architecture de la base de données

Le schéma relationnel suit fidèlement le modèle imposé par le sujet :

Table	Clé primaire	Clés étrangères	Rôle
Departement	idDep (AUTO_INCREMENT)	–	Référentiel départements
Region	idRegion (AUTO_INCREMENT)	idDep	Référentiel régions
Client	idCli (AUTO_INCREMENT)	idRegion	Clients (photo, email unique)
Produit	idProd (AUTO_INCREMENT)	–	Catalogue produits
Commande	idCmd (AUTO_INCREMENT)	idCli	En-tête de commande
ligneCommande	(idCmd, idProd)	idCmd, idProd	Détail : quantité par produit

Table 1 – Structure des tables

Choix techniques

— **ENGINE=InnoDB** : support des clés étrangères et des transactions ACID.

— **ON DELETE CASCADE** : la suppression d'une commande supprime automatiquement ses lignes, évitant les enregistrements orphelins.

— **UNIQUE KEY sur email** : l'unicité des clients est garantie au niveau base de données.

— **Charset utf8mb4** : support complet des caractères Unicode (accents, caractères spéciaux).

— **Données de test** : le fichier `schema.sql` insère 5 départements, 5 régions, 5 clients, 8 produits, 5 commandes et 12 lignes de commande pour faciliter les tests.

Architecture applicative

Organisation des fichiers

Pattern architectural

L'application suit un pattern **page-controller** : chaque URL correspond à un fichier PHP qui gère la logique métier et le rendu. Les fichiers `*_save.php` et `*_delete.php` traitent exclusivement les requêtes POST et redirigent via `Location`, appliquant le pattern **POST/Redirect/GET (PRG)** pour éviter les soumissions doubles lors d'un rafraîchissement de page.

Fichier	Rôle
config.php	Connexion PDO, fonctions utilitaires
includes/navbar.php	Barre de navigation partagée
index.php	Tableau de bord (KPIs + graphiques)
produits.php	Liste des produits avec recherche
produit_form.php	Formulaire création/modification produit
produit_save.php	Traitement POST produit
produit_delete.php	Suppression produit
clients.php	Liste des clients avec recherche
client_form.php	Formulaire création/modification client
client_save.php	Traitement POST client
commandes.php	Liste des commandes
commande_form.php	Formulaire nouvelle commande (multi-lignes)
commande_save.php	Traitement POST commande (transactionnel)
commande_detail.php	Détail d'une commande
ligne_delete.php	Suppression d'une ligne de commande
uploads/	Stockage des photos

TABLE 2 – Fichiers du projet

Sécurité

La sécurité repose sur un principe fondamental : *ne jamais faire confiance aux données du client.*

— **Requêtes préparées PDO** : toutes les requêtes SQL utilisent des paramètres liés (`?` ou `:placeholder`), éliminant le risque d'injection SQL.

— **Protection XSS** : la fonction `h()` applique `htmlspecialchars()` avec `ENT_QUOTES` sur toute donnée affichée dans le HTML.

— **Validation serveur** : chaque fichier de traitement revalide les données (champs obligatoires, format email, types numériques) indépendamment de la validation JavaScript.

— **Méthode HTTP** : les opérations de modification utilisent exclusivement POST ; les fichiers de traitement rejettent les requêtes GET avec un code 405.

— **Upload sécurisé** : vérification de l'extension, renommage avec `uniqid()` pour éviter l'écrasement et les attaques par nom de fichier.

Interface utilisateur

Technologies front-end

— **Bootstrap 5.3** : framework CSS responsive, garantissant un affichage correct sur mobile et desktop via le système de grille et les composants adaptatifs.

— **Bootstrap Icons** : icônes vectorielles cohérentes dans toute l'interface.

— **Chart.js 4** : bibliothèque de graphiques légère utilisée pour le tableau de bord.

Choix d'ergonomie

— **Messages flash** : notifications temporaires stockées en session PHP, affichées une seule fois puis effacées. Cela évite les messages persistants après un rafraîchissement.

— **Calcul en temps réel** : dans le formulaire de commande, les sous-totaux et le total se recalculent instantanément via JavaScript à chaque modification de produit ou quantité.

— **Badges de stock** : code couleur vert (> 10), orange (≤ 10), rouge ($= 0$) pour repérer rapidement les ruptures.

— **Confirmation de suppression** : toute action destructrice demande une confirmation utilisateur via `confirm()`.

— **Navigation unifiée** : barre de navigation partagée avec détection automatique de la page active.

Tableau de bord – Analyse de performance

Le tableau de bord (page d'accueil) répond à la tâche 5 du sujet en proposant une analyse multi-axes :

1. **4 cartes KPI** : nombre de produits, clients, commandes, et chiffre d'affaires total. Offrent une vue synthétique immédiate.
2. **Graphique CA/mois** : diagramme à barres combiné avec une courbe du nombre de commandes (axe double). Permet de corréler le volume de commandes avec le revenu généré et d'identifier les tendances.
3. **Top 5 produits** : diagramme en anneau (doughnut) montrant les produits les plus commandés en quantité. Utile pour orienter les décisions d'approvisionnement.
4. **Commandes récentes** : tableau des 5 dernières commandes avec nombre de lignes et montant.
5. **Alertes stock faible** : produits dont le stock est ≤ 10, badge rouge pour stock à zéro. Permet d'anticiper les ruptures.
6. **CA par client** : classement des clients par chiffre d'affaires généré avec nombre de commandes, permettant d'identifier les meilleurs clients.

Cette approche permet une analyse de performance sur quatre axes : *temporel* (tendance mensuelle), *produit* (popularité), *client* (valeur), et *logistique* (gestion des stocks).

Gestion des transactions

La création d'une commande implique deux opérations :

1. Insertion de l'en-tête dans la table `Commande`
2. Insertion de chaque ligne dans `ligneCommande`

Ces opérations sont encapsulées dans une **transaction** (`beginTransaction` / `commit` / `rollBack`), garantissant l'**atomicité** : soit la commande complète est enregistrée, soit rien n'est modifié en cas d'erreur. Cela évite les commandes sans lignes ou les lignes orphelines.

Instructions de déploiement

1. Importer `schema.sql` dans phpMyAdmin avec les identifiants fournis (Login : `MasTMound2`, Mot de passe : `MoundMasT2@2026`).
2. Placer le dossier `tpexam1` dans le répertoire `htdocs` du serveur web (XAMPP, WAMP, etc.).
3. Vérifier que le dossier `uploads/` est accessible en écriture par le serveur web.
4. Ouvrir `http://localhost/tpexam1/` dans le navigateur.